Intro to Data Analytics INFO3406 Exam notes.

Charles Christopher Hyland

Semester 2 2017

Abstract

Welcome! Hopefully these notes help you to ace INFO3406! (But don't ever take this unit too)

Contents

1 Introduction to Data Analytics **1**
1.1 Metrics . . . 1

2 Decision Trees **2**
2.1 Learining . . . 4

3 Model Evaluation **4**

4 Logistic Regression **5**

5 Clustering **5**
5.1 Introduction . . . 5
5.1.1 Types of Clusters . . . 5
5.2 K-means Clustering . . . 6
5.2.1 Bisecting K-means . . . 8
5.2.2 Processing . . . 8
5.3 Hierarchical Clustering . . . 9
5.3.1 Agglomerative/Bottom up . . . 9
5.3.2 MIN/Singe Link . . . 9
5.3.3 MAX/ Complete Link . . . 9
5.3.4 Group Average . . . 10
5.3.5 Ward's Method . . . 10
5.3.6 Application of K-means . . . 10
5.4 Density-Based Clustering . . . 10
5.5 Cluster Validity . . . 11

6 Pattern Discovery **12**

7 Text Mining **15**
7.1 Sentiment Analysis . . . 15
7.2 Applications of Pattern discovery . . . 16

8 Evaluation **16**

9 Data Visualisation **16**

1 Introduction to Data Analytics

2 types of data, structured and unstructured data. 4 V's of data:

Volume: size of data

Variety: what form does it come in? Structured/unstructured/text/multimedia.

Velocity: How fast is it coming in?

Veracity: How reliable is this data.

We have a large number of data that is **dark data** which is hidden within networks, people, and machines.

CRISP-DM There are 6 steps

Business Understanding Understand the business objectives. Assess the current systems. Break down the tasks required. Identify constraints in the projects. Build a project plan (alongside assumptions/risks).

Data Understanding Collect Data. Are there any legal/technical issues to getting this data? Data description and document the data quality. Explore the data.

Data Preparation Integrate data from multiple tables and summarisation. Select data. Data transformation. Clean the data. Construct new data.

Data Modelling Select the appropriate modelling technique. Develop a testing regime. Build a model. Assess the model.

Evaluation Validate the model. Review process. Determine next steps.

Deployment Knowledge deployment is specific to objectives. Process deployment/production. Produce a final report.

1.1 Metrics

Mahalanobi distance takes the co-variance into account and the scales of the variables whilst Euclidean doesn't.

The Euclidean distance ,as you should know from high school, is simply find the hypotenuse of the triangle. However, this assumes that the variables are of the same unit and also that they are uncorrelated.

The Mahalanobis does take the scale and correlation into account in it's calculation.

2 Decision Trees

When determining on how best to split the data, we can look at the distribution of the response variables before and after splitting. We denote p(i) as the fraction of records belong to class i at a given node. We want to split the data to get pure partitions (that is, having $p_1 = 1$ means that we only have instances of label 1 in that split). Impurity tells us how we should split the data. The lower the impurity, the better the split of the dataset. Therefore, we compare the impurity before and after the split, and if that difference is large, then the split was a good split.

Split metrics include:

$$Entropy(t) = -\sum_{i=0}^{c-1} p(i|t) log_2 p(i|t)$$

$$Gini(t) = 1 - \sum_{i=0}^{c-1} [p(i|t)]^2$$

$$ClassificationError(t) = 1 - max_i[p(i|t)]$$

whereby c is the number of classes and $0log_2 0 = 0$ in entropy calculations. The less imbalanced the classes are, the metrics will be higher.

Example If we had 2 classes where class 0 had a count of 1 and class 1 had a count of 5, we would have:

$$Giniindex = 1 - (\frac{1}{6})^2 - (\frac{5}{6})^2 = 0.278$$

$$Entropy = H(X) = -\frac{1}{6} log_2(\frac{1}{6}) - \frac{5}{6} log_2 \frac{5}{6} = 0.650$$

$$Error = 1 - max[\frac{1}{6}, \frac{5}{6}] = \frac{1}{6}$$

$$\chi^2 score = \chi^2(X) = \sum_{i=1}^{k} \frac{o_i - e_i}{e_i}$$

For all these, the lower the impurity values, the better the split of the dataset. We need to compare the difference of the impurity and a larger difference means a better split. This is known as the **gain** Δ. When we look at the difference in *entropy*, then it is known as the **information gain**.

Gini split examines the gini index but computing things out of how the mini datasets constructed by the split. We then take the weighted average of the Gini based on how many observations go into what dataset. From this, we can parititon the dataset on different

features and compute the Gini split score for each of them. The best partition is given by a lowest Gini-split score. If we had continuous variables, sort the attribute on the values and then create numerous if else conditions whereby we paritition the dataset into 2 based on different baseline values to use. Then compute Gini split and pick the level of the continuous variable which is best to split on.

Gini Gain/informatino gain is similar to the gini split as it is the: GiniGain(S,X) = Gini(S) - GiniSplit(X), whereby S is the original node and X is the attribute used for the separation. The best partition is given by the highest gini gain. We look at the difference between the entropy of the parent less the entropy of the children.

Entropy looks at the amount of randomness of probability distribution. We can repeat and analyse the split using the entropy metric in the same way we used the Gini metric. We can look at the information/entropy gain to look at the reduction in entropy because of the split:

$$H_{\text{gain}}(S, X) = H(S) - H_{\text{split}}(X)$$

The best partition is given by a higher H_{gain}. However, an issue is that it tends to prefer splits that result in large number of partitions, whereby each one is small yet pure.

Classification error is Error(X) = 1 - maxP(x). When we split a node, we can also analyse the error split and error gain like in the previous examples.

We also have the χ^2 score to test independence between 2 categorical attributes (such as the feature and the class label). For that, we have the counts in a contingency table and we calculate the normalized squared deviations of observed less expected(actual) values:

$$\chi^2 = \sum_{i=1}^{k} \frac{(o_i - e_i)^2}{e_i}$$

We have both a contingency table and an expected table, whereby the entry in the expected table $eij = \frac{o_i}{N} \dot{o_j} N$.

In other words to compute this $E_{\text{High,Buy}}$ on slide 54 of lecture 6, we go:

$$\frac{o_i}{N} = \frac{2+2}{14} = 0.2857$$

$$\frac{o_j}{N} = \frac{2+4+3}{14} = 0.643$$

Then to compute the expected value for $E_{\text{High,Buy}}$

$$E_{\text{High,Buy}} = 0.2857\dot{0}.643\dot{1}4 \approx 2.57$$

Then, we can compute the $\chi^2 = \sum\limits_{i=1}^{k} \frac{(o_i - e_i)^2}{e_i}$.

There are different degrees of freedom for the different features and therefore we need to compare the probability of obtaining χ^2 score in each distribution. Recall that the degree of freedom is df = (c-1)(m-1) whereby m is the number of values for attributes and c is the number of classes. So for example, if we had y = [buy,no buy] and feature is [high, medium, low], the degrees of freedom would be $\chi_{(2-1)(3-1)}$. From this, we can compute the p-value and we choose the p-value that is the lowest (WE CAN'T COMPARE THE χ^2 SINCE DIFFERENT DEGREES OF FREEDOM).

2.1 Learining

We can try brute force approach for all trees and it is guaranteed to find the best model. However, this is computationally infeasible. We can use **heuristic searches** and select a local optimum. One such example is a **greedy search**. This is a top-down recursive divide and conquer algorithm whereby we partition by current attributes and then pick the one with the best result. The issue is though is how to construct the features, when to stop growing, and how to prune irrelevant parts of the tree. We prune trees to avoid overfitting.

Optimistic Approach: Look at the number of differences (that is, you can just take the min of the 2 labels).

3 Model Evaluation

Accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$

Precision = $\frac{TP}{TP+FP}$

Recall = $\frac{TP}{TP+FN}$

F-Measure = $\frac{2TP}{2TP+FN+FP}$

$$\text{Weighted Accuracy} = \frac{w_1TP + w_4TN}{w_1TP + w_2FN + w_3FP + w_4TN}$$

We can create cost matrices out of confusion matrices. We want to minimise these cost matrices.

4 Logistic Regression

To get how much the odds are multiplied by every unit increase, we go $\exp(\hat{\beta}_1)$. Thus, if we increase X by 1 unit, we multiply the odds by $\exp(\hat{\beta}_1)$.

5 Clustering

5.1 Introduction

Cluster analysis is finding groups of objects such that the objects in a group will be similar to one another whilst being different to objects from other groups. We minimise **intra-cluster distances** whilst maximizing **inter-cluster distances**. This can help us with understanding and summarizing data.

Clustering is not supervised classification, simple segmentation, results of a query (since we have a specification), or graph partitioning.

Clustering relates to **unsupervised learning**. There are 2 methods in clustering: partitional clustering and hierarchical clustering. **Partitional clustering** is where we divide objects into one subset initially whilst **hierarchical clustering** is a set of nested clusters organised as a hierarchical tree. We can then use a *dendogram* to examine the hierarchical clustering.

4 important terminologies to distinguish between sets of clusters: **Exlusive/Non-exclusive:** In non-exclusive clusterings, points may belongs to multiple clusters. This helps to represent observations that are on the border of classes and could be between 2 different classes.

Fuzzy/Non-fuzzy: For each point, we assign a weighting to each cluster from 0-1. Sum of weightings for each points must = 1. If it doesn't belong to that cluster, weight is 0. **Partial/Complete:** We may only want to cluster some of the data. **Heterogenous/Homogenous:** The clusters are of different sizes, shapes, and densities.

5.1.1 Types of Clusters

Furthermore, there are more types of clusters in which we can define:

Well separated clusters: Each point is closer to all the points in its own cluster compare to other clusters.

Center-based clusters: Each point is closer to the center of its own cluster compare to other clusters.

Contiguity-based clusters: Each point is closer to at least ONE point in its cluster compared to points in other cluster.

Density-based cluster: Clusters are regions of high density separated by regions of low density.

Objective Function: Find clusters that optimises an objective function. We look at all different ways to evaluate each potential set of clusters using objective function. *Hierarchical clustering* algorithms has local objective functions whilst *partitional algorithm* has global objective functions. We can also have a variation of the global objective function by fitting the data to a parameterized model (whereby parameters are determined from the data). We can alo map the clustering problem to a different domain and solve it via there. One such way is by using a **proximity matrix** using a weighted graph whereby nodes are the points and weighted edges are the proximities between points. We can then break the graph into connected components, which is equivalent to clustering. We want to then minimise the edge weight between clusters and maximise edge weight within clusters.

Answer we get won't be the optimal one, just a good answer. Running same model on same data, will give us different answers.

Now we look at 3 types of clustering algorithms: **K-means**(partitional), **hierarchical**, and **density-based** clustering.

5.2 K-means Clustering

Each cluster is associated with a **centroid** or center point. We pick random points in the data to be the centroid and see which the points are the closest. We then assign points to the cluster with closest centroid. We repeat this multiple times until centroids don't change. *Note that the number of clusters K must be specified in advance for this algorithm to work.*

So the algorithm is:

K-means clustering:
Select K points as the intial centroids
while Centroids don't change **do**
    Form K clusters by assigning all points to the closest centroid.
    Recompute centroid of each cluster.
end while

Note that the initial centroids are often chosen randomly, which leads for clusters produced to vary in each time we run the algorithm. We define the *centroid* to be the mean of the points in the cluster. We can also define **closeness** as measured by the Euclidean distance, cosine similarity, correlation, and more. K-means will tend to converge for common similarity measures and usually occurs in the first few iterations. The complexity of this algorithm is $O(n\dot{K}\dot{I}\dot{d})$ where n = points, K = no. of clusters, I = number of iterations, d = no. of attributes.

First iteration, we pick random points to be our centroid and construct our clusters. We set the points to be of the same cluster as the closest centroid to it. The centroid moves in the average of the distances of the points. On each iteration, we move the centroids based on the average distance of the points. If cluster is empty, we delete that cluster. We should run the cluster algorithm multiple times. It's very important for the location of the initial centroids since if it's placed in the wrong spot, the model will fall apart.

The most common measure to evaluate the K-means clusters is the **sum of squared error (SSE)**. For each point, the error is the distance to the nearest cluster. From this, to get the SSE, we square the errors and sum them:

$$SSE = \sum_{i=1}^{K} \sum_{x \in C_i} dist^2(m_i, x)$$

whereby x is a data point in cluster C_i and m_i is the representative point for cluster C_i (normally the center/mean of the cluster.) From this, given two clusterings, we choose the clusters with the smalelst error. We can reduce SSE simply by increasing the number of clusters K (however, a good clustering with a smaller K can outperform a clustering with a higher K).

We can select 20 clusters and slowly decrease the clusters. If this error rate falls, then it's a good idea to keep decreasing the number of clusters.

We can run k-means multiple times in hopes of not suffering from too many issues of the initial centroids problem. Try and get a deterministic solution. We can also choose more than k initial centroids and then choose the centroids that are most widely separated.

The algorithm is known to collapse on local minima instead of the global due to the random initilisation in the beginning. What we can do is post-processing of the created clustered model, whereby we can split clusters up based on which cluster has the highest SSE. The rationale is that the child clusters will have a lower SSE compared to the parent cluster. This leads to the next variant of K-means.

5.2.1 Bisecting K-means

This is a variant of K-means that produces a partitional or a hierarchical clustering. First, we initially set all points to be in the same cluster. We still need to know how many clusters K we want in the end, so that still needs to be specified (and the algorithm will run until we get these K clusters). Then, we select a cluster, and call the basic K-means algorithm on it, whereby we form 2 clusters randomly and assign points to the 2 new centroids. We repeat this as many times as we want on the same cluster. We then choose the pair of child clusters from the bisection with *lowest SSE* to be added to the list of clusters.

Bisecting K-means
Initialise a list L of clusters which contains all points in one single cluster.
while Number of clusters $< K$ **do**
    Pop a cluster c from L
    for $i = 1$: specified iterations **do**
        Bisect the cluster c using K-means
    end for
    Add 2 children clusters with lowest SSE to L
end while

5.2.2 Processing

Pre-processing: We should normalise the data and remove outliers. Outliers will greatly affect the centroids and their movements. This will help reduce the error.

Post-processing: Here, we eliminate small clusters that represent outliers. We can also split loose clusters with relatively high SSE. We can also merge clusters that are and have low SSE. We can use any of these steps during the clustering process.

There are a few limiations of K-means:

Differing size: If clustering sizes differ, then this is an issue since this causes points in bigger clusters to be attracted to other centroids. This algorithm tends to create balanced clusters.

Density: Clusters can also pick up the wrong points if not dense. We can pick up wrong spots due to weird densities and tricking the algorithm.

Non-globular shapes: Clusters again can pick the wrong clusters by picking up parts from other clusters.

Outliers: Hard to cluster when there are outliers.

A solution is to use many clusters and then combine them ourselves.

5.3 Hierarchical Clustering

This produces a set of nested clusters organised as a hierarchical tree. We have a hierarchy in points. We have points that can be subsets of other clusters. We can visualise it with a **dendogram** which records the sequences of merges/splits.

Advantages are that we don't need to assume apriori number of clusters and they can also correspond to meaningful hierarchies. We measure the distance in different ways. Instead of using centroids, we use a hierarchy. We use similarity measures such as Gini.

2 main types of hierarchical clustering: **Agglomerative** starts with points as individual clusters and merge closest pair of clusters until only one cluster (or k) is left. **Divisive** Starts with one all-inclusive cluster and at each step, split the clusters until we have n cluster (or k if we specified that). We use a similarity/distance matrix to help decide on how to split.

5.3.1 Agglomerative/Bottom up

Agglomerative Clustering
Compute Proximity Matrix
Let each data point be a cluster
while We have more than a single cluster **do**
    Merge two closest clusters
    Update proximity matrix
end while

The key operation is the computation of the proximity matrix and computing the proximity of two clusters. How do we merge clusters? We have 4 solutions to it. We want to look at the distances between centroids.

5.3.2 MIN/Singe Link

We look at the 2 most similar (closest) points in the different clusters. Looking at the minimum on page 58, looking at minimum, it is 3 and 6. Then 2 and 5. Then 2,5 and 3,6 are the closest cluster. The strength is that it can handle non-elliptical shapes. However, it is still sensitive to noise/outliers

5.3.3 MAX/ Complete Link

Maximimum We look at the 2 least similar (furthest) points in the different clusters. Looking at maximum on page 62, we look at the minimum of the maximum point distance.

This is less susceptible to noise and outliers but is biased to globular clusters. It tends to break these globular clusters as it spills over into other clusters.

5.3.4 Group Average

Group Average: We take average of distances and then look for the minimum.

$$\text{proximity(Cluster i, Cluster j)} = \frac{\sum\limits_{p_i \in imp_j \in j} \text{prox(pi,pj)}}{Clusteri * Clusterj}$$

Group average is a good mix between minimum and maximum. Less susceptible to noise and outliers but still biased towards globular clusters.

5.3.5 Ward's Method

Ward's Method: We use a score instead of a distance to merge clusters. We look at the similarity of the cluster by the increase in the squared error when the 2 clusters are merged. It is similar to group average except we now square the distances. Less susceptible to noise and outliers but still biased towards globular clusters. Quite similar to K-means and can be used to initialise K-means.

Understand how the methods differ and the results they give us. What are the advantages and disadvantages of each method.

Once we decide to combine 2 clusters, we cannot reverse our decision. We also don't have an objective function to directly minimise.

5.3.6 Application of K-means

We can use it for **color quantization**. This is where we reduce the number of distinct colours used in an image whilst ensuring the new image is as visually similar as possible to original image.

5.4 Density-Based Clustering

DBSCAN is the density-based algorithm. We define density as the number of points within a specified radius (Eps). Given a set of points in some space, it groups together points that are closely packed together (points with many nearby neighbors), marking as outliers points that lie alone in low-density regions (whose nearest neighbors are too far

away). Good for unusual shapes. 2 different things in this algorithm: distance between points (Eps) and the minimum number of points required to be classified into a certain category for 3 different categories (a point can be a border point, core point, or a noise point).

The minimum points is the number of points needed to be inside the circle (Eps) and based on that, we can classify the point. If a point has the minimum number of points required, then it will be known as a **core point**. If a point has no points inside its radius, then it'll be known as a **noise point**. A **border point** is a point with more than 0 points but less than the minimum number of points required to be a core points. It is resistant to noise and can handle clusters of different shapes and sizes.

Having sparse data is bad and effects it. Furthermore, high-dimensional data will affect it too. On page 76, hierarchical clustering will be better.

Our issue is finding the optimal hyperparameters. We want that for points in a cluster, their kth nearest neighbours are roughly the same distance. We can plot the sorted distance of every point to its kth nearest neighbour. If EPS is high, then all points will be into 1 cluster on page 77. If Eps too small, points won't be placed into correct clusters. We can find EPS by looking for the **knee** of the graph. We can set Min points based on number of dimension of data + 1.

5.5 Cluster Validity

Very hard to measure whether are the clusters good or not. We can't really evaluate clusters properly but we still want to so that we can avoid patterns in noise, to compare cluster algorithms, to compare sets of clusters, or even clusters themselves.

When determining the **clustering tendency** of a set of data, we distinguish whether does a non-random structure actually exist in the data. We compare the results of a cluster analysis to externally known results and see how well our cluster analysis does.

We can judge clusterings or clusters with these **indices/criteria** by:

External Index: Measure which cluster labels match externally supplied class labels. Such metrics used is entropy.

Internal Index: Measures good without external information. Such metrics used is sum of squared error. We can also use SSE to estimate the number of clusters.

Relative Index: Compare two different clusterings or clusters. We can use both internal or external for this.

Example at end of lecture slides, be able to interpret the output of the table.

6 Pattern Discovery

Suppose we are in a supermarket and people make purchases. One of the things we can do is to see if we can extract patterns and see whether what items are related. Given a set of transactions, find rules that will predict the occurrence of an item based on the occurrences of other items in the transaction. So if someone buys diaper, they are likely to buy beer too. From this, given a set of transactions, find rules that will predict the occurrence of an item based on the occurrences of other items in the transaction. This is co-occurrence, *not causality.*

We can also derive rules whereby based on the characteristic, we can set rules to help classify things. Furthermore, we can see if patterns makes sense and if not, then that means we are discovering new things. For this, we need to do some data transformation by creating dummy variables for a matrix, to indicate whether was a good bought in a particular transaction.

Primitive pattern are subsets of all possible observations over variables $X_1, ..., X_p$. If X_k is categorical, then $X_k = c$ and therefore is a primitive pattern. We can also have rules as well. We can also combine rules together whereby it combines different rules. So if we had number of beers bought, we can convert that into whether did someone buy or not. If we had ordinal data, we can convert it into a range query. We can then start from primitive patterns and combine using logical connectives such as AND/OR.

Item set: Different kind of items in our dataset. These are our subsets of our data. We call this a k-itemset (itemset that contains k items).

Support count σ: The frequency of an occurrence in an itemset. This counts the occurence of a particular combination of items (how many times does a combination of items appear).

Support s The fraction of transactions that contains the combination itemset. So support count tells us the frequency of transaction whilst support tells us the fraction of transaction

Frequent Itemset: Itemset whose support is greater than or equal to a minsup (minimum support) threshold.

Association Rule: Implicit expression of the form X $\rightarrow$ Y where X and Y are itemsets.

T stands for transactions.

Confidence c: Measures how often items in Y appear in transactions that contain X. So if we had itemset Y, how many times does it appear whenever itemset X appears.

We have 5 different transactions and 4 different items. From this, we can construct these trees for SUPPORT and CONFIDENCE. Each vertex tells us the frequency/support of that itemset in the transactions.

2^d possible itemsets.

We have a threshold for minimum support and minimum confidence (minsup and minconf). Support tells us frequency of itemset and confidence tells us how confident we are that it appears. We try to achieve a minimum threshold such that if a rule falls below a threshold, we can remove that rule. If we have a very large database, then support and confidence will be very small since we have so many items (as they are just a fraction of all items). We can do a Brute-force approach to get all rules and prune rules. However this takes forever.

What we can do is a 2 step approach. First, generate all itemsets whose support $\geq$ minsupport. Then we generate high confidence rules from each frequent itemset, whereby each rule is a binary partitioning of a frequent itemset. This is still quite computationally expensive to generate all frequent itemsets.

Apriori principle: We can use this to reduce number of candidates. If itemset is frequent, then all of its subset must also be frequent. So if milk,eggs,cheese are frequent, then subsets of that must be frequent. This principle holds due to the property of the support measure:

$$\forall X, Y : (X \subseteq Y) \rightarrow s(X) \geq s(Y)$$

whereby the subsets X of Y, the support of X is at least the support of Y. This is the **anti-monotone** property of support.

This holds vice versa and so, if we find an infrequent itemset, we can prune it to remove all the childs of that itemset too. We don't need that rule.

We can use the **Apriori algorithm** to create rules. Efficiently search for rules that meet support/confidence thresholds.

1. Set k = 1, and determine list of candidates of size k
2. Calculate frequency of the set
3. Prune sets with low frequency based on a support threshold
4. Frequent sets of size k are combined to generate sets of size k+1
5. If there exists more than a single frequent set of size k, go back to step 2

Use frequent itemset to form association rules

We set a min support and min confidence and compute the tree from there.

Association rules return many rules. However, alot of rules are not needed. We want to quantify whether the data is useful by looking at it objectively (statistical measures) or subjectively (requires domain knowledge).

Given rule $X \to Y$, compute statistics based on contigency table (which is similar to a confusion matrix). There are 6 different measures. We have item X, Y, not X, and not Y. Tells us the frequency combination of the 2 itemset.

Support If a supermarket sells a car, then even though that transaction is quite rare, when it does sell, this leads to a high surce in revenue. However, support won't take that into account due to the infrequency of cars being sold. We can look at an example, people tend to buy EITHER tea or coffee. From this table, we can list people who buy coffee and tea, people who buys just one of them, and people who buy neither of them.

$$P(X|Y) = \frac{f_{1,1}}{f_{\sum Y}}$$

$$P(X) = \frac{\sum X}{N}$$

Lift tells us the independence between 2 items. If lift is around 1, then most likely there is no rule. Lift is just confidence over support. Goes from $0 \to \infty$. If less than 1, then negative association and if greater than 1, positive association. Higher lift tends to suggest rule is potentially useful for future.

$$Lift(X,Y) = \frac{P(Y|X)}{P(Y)} = \frac{c(X \to Y)}{s(Y)}$$

Conviction tells us the ratio of expected frequency X occurs without Y. Measures the degree of implication of a rule. If conviction is near 1, then most likely no rule. Else, if it's greater than 1, seems to suggest there is a rule.

$$Conv(X,Y) = \frac{P(X)P(\bar{Y})}{P(X,\bar{Y})} = \frac{s(X)s(\bar{Y})}{s(X,\bar{Y})}$$

Piatetsky-Shapiro/Leverage: How much more counting is obtained from co-occurence of antecedent and consequent from expected. Varies from (-0.25,0.25). Close to 0 indicates good rules.

$$PS(X,Y) = P(X,Y) - P(X)P(Y) = S(X,Y) - s(X)s(Y)$$

Correlation analysis: Should be close to 1 to be considered good. Analysing relationship between pair of variables.

$$\phi(X,Y) = \frac{P(X)P(Y) - P(\bar{X},Y)P(\bar{Y},X)}{\sqrt{P(X+) + P(Y+) + P(\bar{X}+) + P(\bar{Y}+)}}$$

$$\phi(X,Y) = \frac{f_{11}f_{00} - f_{01}f_{10})}{\sqrt{f_{1+}f_{+1}f_{0+}f_{+0}}}$$

Here, we can just multiply by the numbers of what's actually inside. Slide 29 = 0.25

Is measure Cosine measure for binary variables. Varies between 0 and 1.

$$IS(X,Y) = \frac{P(X,Y)}{\sqrt{P(X)P(Y)}} = \frac{s(X,Y)}{\sqrt{s(X)P(Y)}}$$

If IS = 0, then no overlap. If around 1, then strong association.

7 Text Mining

We have more data that is unstructured that is structured. Lots of things such as emails, forms, and other mediums tend to have alot of data. However, it is difficult to extract data from all this.

What we normally try to do is extract words whilst removing stoppage words and compute the frequency of each word. We can look for a particular word such as Israel and see what words are associated with Israel and see the frequency of the words. Tells us the view of the company.

Wordcloud are where the bigger the word, the more frequent it appeared in the data.

7.1 Sentiment Analysis

We want to know what the sentiment/feelings are about certain thing. We take the text, cut it into pieces and put it into a dictionary. We have 2 different dictionary where one is for good words and other for bad words. Then we compare the frequencies of the word from the respective dictionary. If we have positive words, then what we have to do is assign a score/value to each word in the dictionary. For example, if a word has strong good conotations, then we give it a higher score. We do the same vice versa for negative words. We can compute the scores of the text based on number of good/bad words. A higher score means a more positive text. We can then plot it out on a graph. You should also report the number of mentions of the subject we are looking at, alongisde the sentiment score of it.

Another way to do this (not sentiment analysis but classification). We take the news and vectorize it by transforming it into dummy variables. We then give it new news and see how it reacts to it. We can do a vectorization whereby column tells us the presence of a word. Meanwhile the row is for a particular observation. At the end of the row, we have a classification on whether it is a positive or negative sentiment. From this, we have a model whereby the more words we have, the bigger the tree. From this, we can do classification for sentiment analysis via tokenization.

7.2 Applications of Pattern discovery

Let us say we have mode some rules and we want to know how to help add value to our company. From this, we can do these two things:

Cross selling: When they buy something and you also offer them a complementary product. When you buy a burger, they also offer you french fries and a coke.

Up selling: Paying extra to get a larger size french fries.

Product layout: Pricing the product.

Product traffic: Placing the items in the market to force them to go certain directions.

Identify customer behaviours We can do special campaigns towards customers based on their purchasing habits. If we know behaviour of what customers are buying, you can make offers and bundles for them and offer promotions.

8 Evaluation

How can we evaluate the different models and also how good is the model we have with regards to the business understanding.

9 Data Visualisation

Memorise: Colours, line/forms, typography, format/scale, icons, images. We aim to communicate complex ideas with clarity, precision, and efficiency. How to make data readable. We want to allow the user to understand what the data means.

We need know what is our question and what is the message we wish to deliver. There's a few things to consider when graphing data: colours, lines/forms, typography, format/scale, icons, images, and more. Try not to use colours that vibrate (too bright and clash). Careful of people that are color blind. Don't use pie charts.

Don't overuse colours. Now graphs are becoming more static and being able to be interacted with.